Intelligent Methods for Resilience: Household Energy Shifting in Western Denmark (DK1)

Problem

The DK1 electricity system is increasingly challenged by the combination of rising electricity demand and a growing share of variable renewable energy. This leads to periods of increased system stress, where demand places greater pressure on the grid, which can lead to congestion, inefficient use of renewable energy, and reduced system stability. Meanwhile, a portion of household electricity consumption is inherently flexible, meaning it can be shifted in time without significant loss of utility. The core challenge is therefore not only technological, but behavioral and informational: **how to exploit this latent flexibility in a way that meaningfully supports system resilience.**

Question

How can price forecasting and explainable energy recommendations be used to reduce grid stress and improve resilience in the DK1 electricity system?

Method

The project combines XGBoost-based electricity price forecasting with uncertainty and explainable AI methods, and translates these forecasts into actionable household recommendations. Finally, behavior is modelled through a 120-hour rolling shifting heuristic to simulate the resulting resilience impact.

Underlying idea

The underlying idea is that electricity price forecasts can help households shift flexible consumption away from hours where demand is expected to place greater pressure on the grid. By translating forecasts into clear recommendations, households can move activities such as EV charging, dishwashing, laundry, and drying to less stressful periods without reducing overall comfort.

Although each household only shifts a small amount of demand, many coordinated shifts can reshape the total consumption curve. **The project therefore tests whether forecast-guided household behaviour can reduce high-load stress hours in DK1 and thereby support grid resilience.**

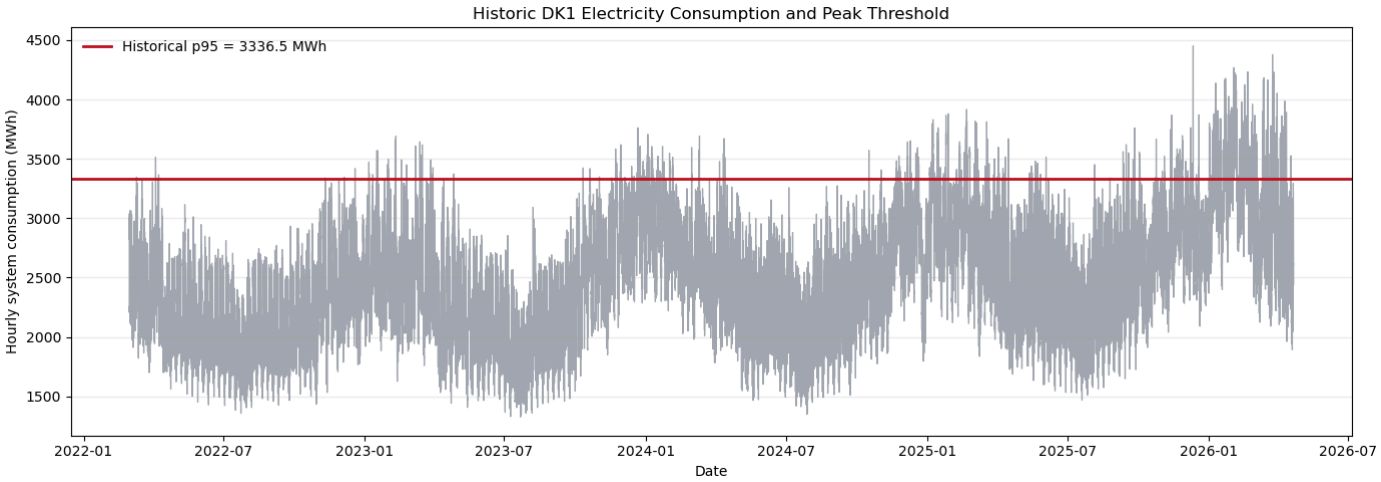
Introduction: Defining Resilience and the Current Grid State in DK1

To evaluate whether the project improves resilience, we first need to describe the current state of the DK1 electricity grid. In this project, electricity consumption is used as the main observable indicator of grid pressure. When consumption is high, larger volumes of electricity must be transported through the network at the same time, placing greater strain on grid infrastructure such as cables, transformers, and substations. Over time, repeated periods of high load can contribute to increased wear, and a greater need for maintenance or reinforcement.

In the absence of asset-level grid data and physical capacity limits, grid stress is defined using a statistical threshold derived from historical DK1 consumption. The first figure below shows the full hourly DK1 consumption series in grey, with the historical 95th percentile marked by a red horizontal line. Hours above this threshold are classified as peak-load hours. These hours are important because they represent periods where the grid is operating at the upper end of its normal demand range. During such hours, network assets are likely to experience greater loads, leaving less operational flexibility and increasing the risk of congestion, accelerated wear, and future reinforcement needs.

```
from src.analysis.energy_congestion_analysis import (
    build_grid_state_summary,
    plot_historic_consumption_with_p95,
)

grid_state = build_grid_state_summary()
historic_consumption_figure, _ = plot_historic_consumption_with_p95(grid_state)
```

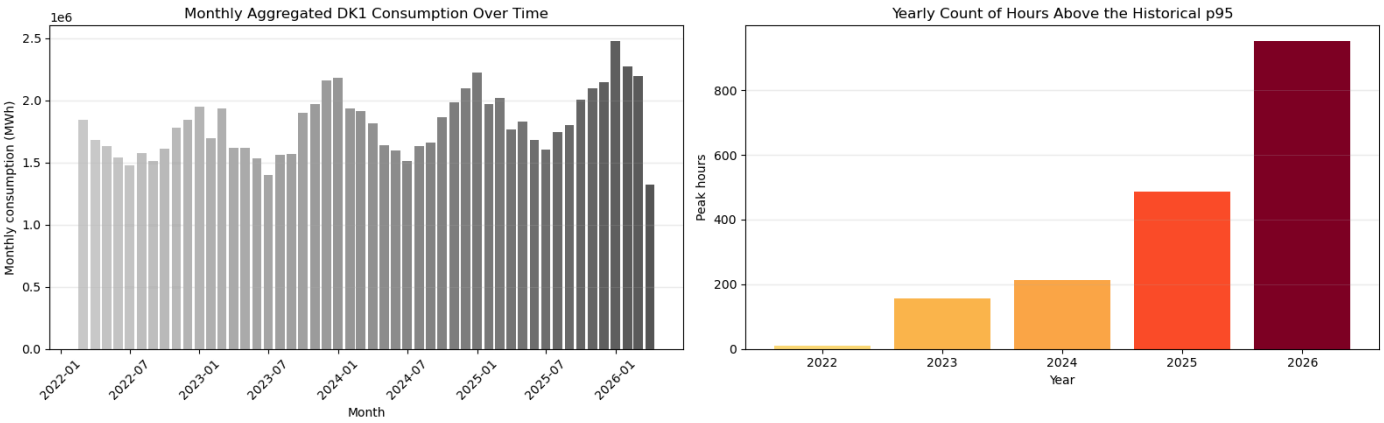


The historic consumption shows that high-load hours are recurring events. It also confirms the upward trend in electricity consumption, with demand increasingly reaching the upper end of its historical load range. To examine this trend more clearly, the next figure aggregates consumption into monthly totals and counts the number of hours in each year that exceed the historical 95th percentile.

The monthly bars make the longer-run demand development easier to read than the raw hourly series. The yearly peak-hour bar chart then serves as a compact snapshot of the current grid state: if total consumption and the number of peak hours both rise over time, the need for action that removes or shifts peak loads becomes more urgent.

```
from src.analysis.energy_congestion_analysis import plot_consumption_trend_and_peak_hours

trend_and_peaks_figure, _ = plot_consumption_trend_and_peak_hours(grid_state)
```



The monthly totals make the long-term increase in demand easier to interpret than the raw hourly series. The yearly peak-hour count shows how grid stress develops over time. As both total consumption and the number of peak-load hours increase, the need to shift flexible demand away from stressed periods becomes clearer. The resilience contribution of the project must therefore be understood as an attempt to improve the timing of consumption by moving flexible demand away from periods where the system is already under pressure.

To achieve this, the project must translate the grid-resilience problem into a signal that households are likely to respond to. Consumers are not expected to shift consumption because of grid stress itself, but household price sensitivity can be used as a practical way to encourage consumption patterns that are indirectly beneficial for the grid. The project uses electricity price forecasts to identify when flexible electricity use is expected to be more or less attractive, and turns this information into guidance for households. Activities such as EV charging, laundry, dishwashing, and drying can then be shifted away from expensive hours and towards cheaper hours. If these price-guided shifts also reduce demand during high-load periods, consumer behaviour is indirectly contributing to lower grid stress and improved resilience.

Forecast Analysis: Predicting hourly electricity prices 120 hours ahead

The forecast is the central analytical element of the project. Energy shifting is a timing decision: a household can only postpone EV charging, laundry, dishwashing, or heating if it has a credible signal that another hour in the near future will be cheaper or less stressful for the system - that is what we are building here. Electricity prices are very useful because they summarize market scarcity, demand, wind and solar production, interconnector conditions, and other system pressures in one observable hourly signal.

The purpose of the forecast is more than minimizing prediction error. It is to make a forward-looking price profile that feeds into a dashboard, which can be turned into meaningfull consumption recommendations. The forecast should not only identify cheap and expensive periods but also the uncertainty around those signals. The recommendation layer can leverage the uncertainty measures to exert confidence when the predicted price difference is large and more caution when the model predictions are noisy. This will build more trust in the system for the household consumers, and ultimately shift more of the flexible consumption.

Data Acquisition: Prices, consumption, weather actuals, and weather forecasts

To forecast DK1 electricity prices, we first need data that describes the price itself and the conditions that influence it. We collect day-ahead prices, observed weather, previous-run weather forecasts, and consumption data. The raw files are saved in `data/`. The consumption data is later used to measure grid stress and simulate load shifting.

Data is collected from two public APIs - Energi Data Service for electricity prices and consumption, and Open-Meteo for historical weather actuals and NWP forecasts.

The main data sources are:

- `data/day_ahead_prices_dk1_raw.csv`: hourly day-ahead prices for DK1 from Energi Data Service.
- `data/consumption_dk1_raw.csv`: DK1 electricity consumption used later to measure grid stress and simulate load shifting.
- `data/weather_actuals_raw.csv`: historical weather observations for the DK1 weather location.
- `data/weather_forecasts_raw.csv`: previous-run weather forecasts from Open-Meteo, used to estimate forecast errors.

All raw datasets can be collected with one call:

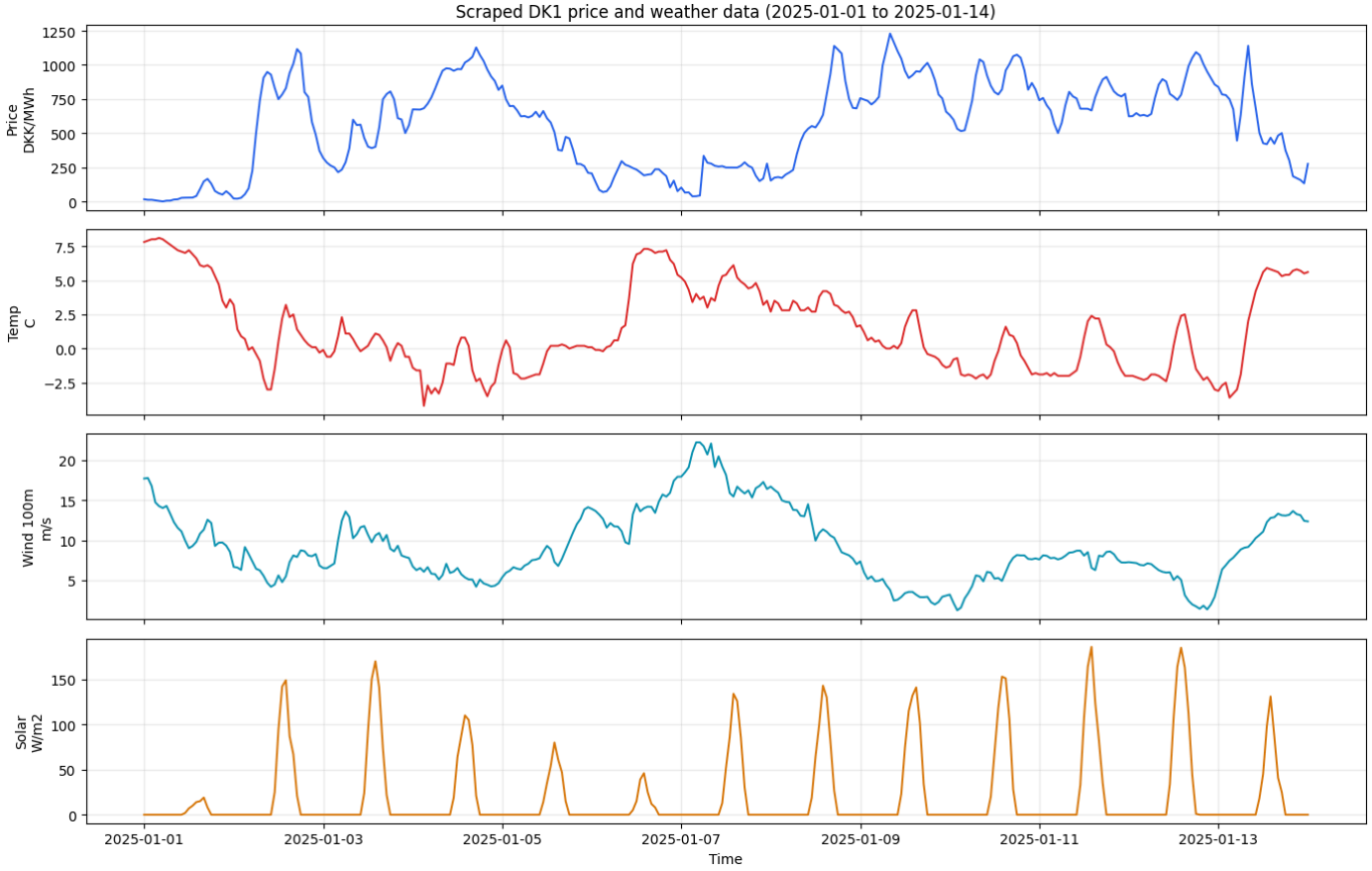
```
from src.data.data_collection import fetch_all

results = fetch_all(start="2021-01-01", end="2026-04-28", price_area="DK1")
```

As a quick check, we plot prices and actual weather for the same period. This helps confirm that the scraped time series line up and that the weather variables are relevant for price forecasting.

```
from src.analysis.forecast_analysis import plot_raw_price_and_weather

fig = plot_raw_price_and_weather(start="2025-01-01", end="2025-01-14")
```



The plot suggests the weather variables are useful predictors of DK1 electricity prices. Price movements appear to line up with changes in wind, temperature, and solar conditions: when wind drops or solar output is low, prices often rise, while stronger wind periods are associated with lower prices. Temperature also seems relevant, as colder periods may increase demand and contribute to higher prices. Overall, the visible movement between price and weather supports the idea that these variables contain predictive information for electricity prices.

Data Processing: Building a leakage-safe forecast matrix

Before modelling, the data must be processed such that it has the same information structure the model would receive in a real forecasting setting. When predicting future electricity prices, the model cannot use future realised observations e.g., about the weather, because those values are not known at the time of prediction. It can only use weather forecasts, and lagged values. This is important to ensure that the model is evaluated under conditions that resemble deployment.

Weather forecast data is difficult to obtain. Historical weather forecasts are available from only from 2025 onwards. This period is used to estimate how forecast errors typically behave across different prediction horizons. These estimated errors are then applied to historical realised weather data to create more forecast-like weather inputs for a longer period of time. The resulting error distributions are saved in:

- `data/weather_error_distributions.csv`:

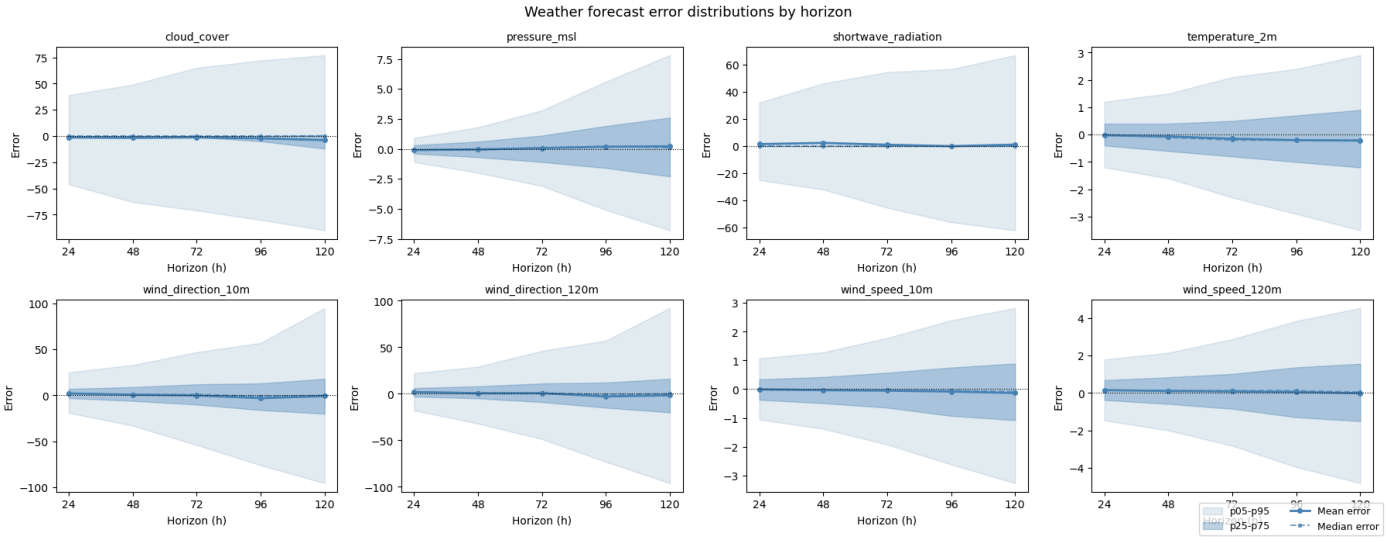
The plot below shows how weather forecast uncertainty changes as the prediction horizon increases. The heteroscedastic behaviour is incorporated into the actual weather data to make it simulate realistic forecasts using the processing script, (`src/data/data_processing.py`). It works by adding noise to the realised weather data using the horizon-specific error distributions. As a result, short-horizon weather inputs remain close to the actual values, while longer-horizon inputs become more uncertain. It builds a forecast matrix with 120 rows for each forecast issue time. Each row represents one prediction horizon, from 1 to 120 hours ahead.

For every issue time and horizon, the matrix contains only information that would be available at the time of forecasting. This includes calendar features, known price lags, and weather variables constructed to resemble forecasts rather than realised observations. The target variable is the realised day-ahead electricity price at the corresponding future target hour.

The final output is a leakage-safe forecast matrix that mirrors the information available in the production dashboard. It allows the model to learn from realistic forecast inputs, rather than from future information it would not have access to in practice.

```
from src.data.data_processing import plot_weather_error_distributions

plot_weather_error_distributions()
```



The figure is an illustration of the five different distributions of forecast errors for key weather forecast variables across increasing longer forecast horizons (24 to 120 hours). For each variable, the mean and median errors remain close to zero, while the spread of the distribution, captured by the percentile bands, widens significantly as the horizon increases.

This indicates that the weather forecasts are largely unbiased, but become progressively more uncertain further into the future. In particular, variables such as cloud cover, shortwave radiation, and wind direction exhibit substantial increases in variability at longer horizons, while temperature and wind speed show more moderate but still noticeable growth in uncertainty.

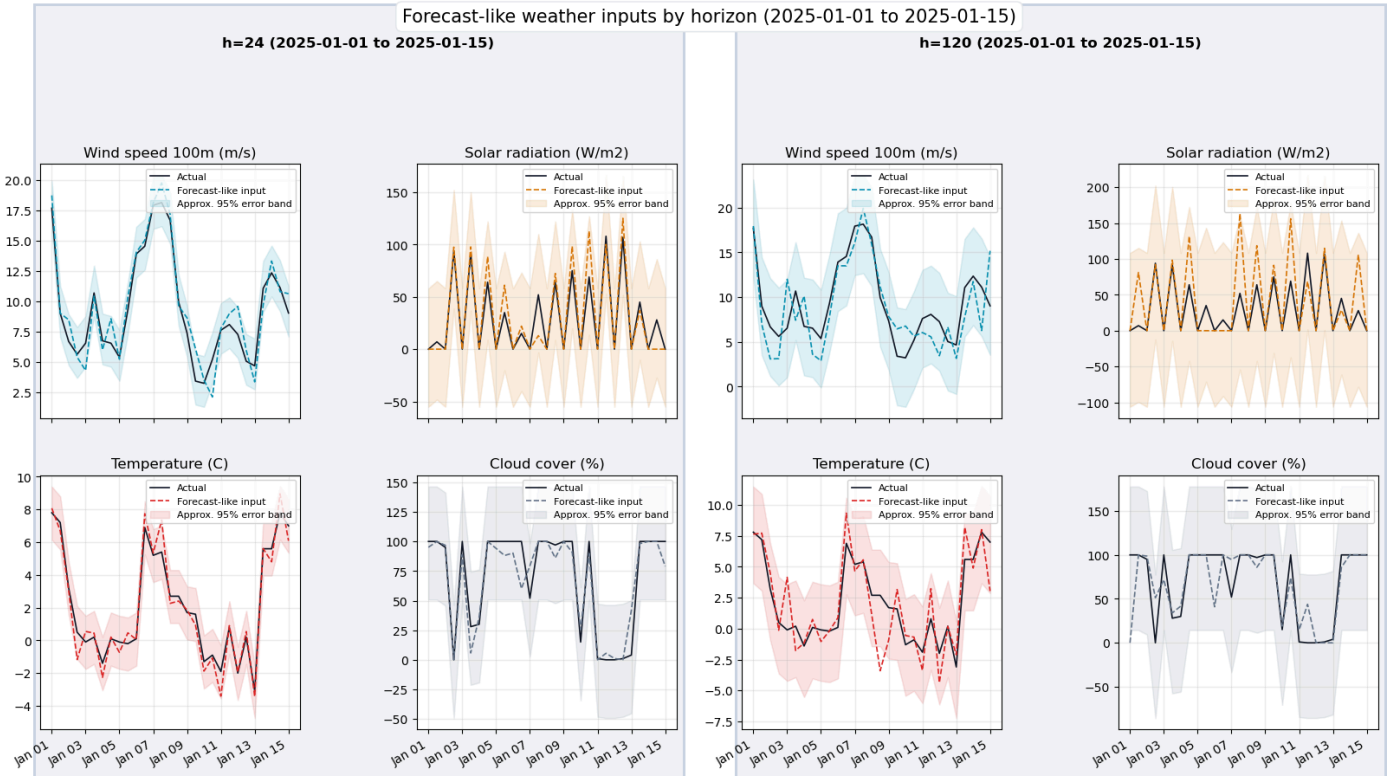
This pattern is important for the electricity price forecasting task. Weather variables are key drivers of both electricity demand (e.g. temperature) and renewable generation (e.g. wind and solar). As forecast uncertainty increases with horizon, the inputs to the price model become less precise, which directly propagates into greater uncertainty in predicted electricity prices. These distributions are the key to create simulated forecast data from realised weather data.

From a modelling perspective, this has two implications. First, it justifies the use of a flexible model such as XGBoost, which can capture nonlinear relationships even under noisy inputs. Second, it motivates the inclusion of forecast intervals rather than relying solely on point predictions, as uncertainty is an inherent and increasing feature of the data.

In the plots below, we can see how the forecast error is applied across different forecast horizons.

```
from src.analysis.forecast_analysis import plot_weather_forecast_grid

fig = plot_weather_forecast_grid(horizon_hours=[24, 120])
```



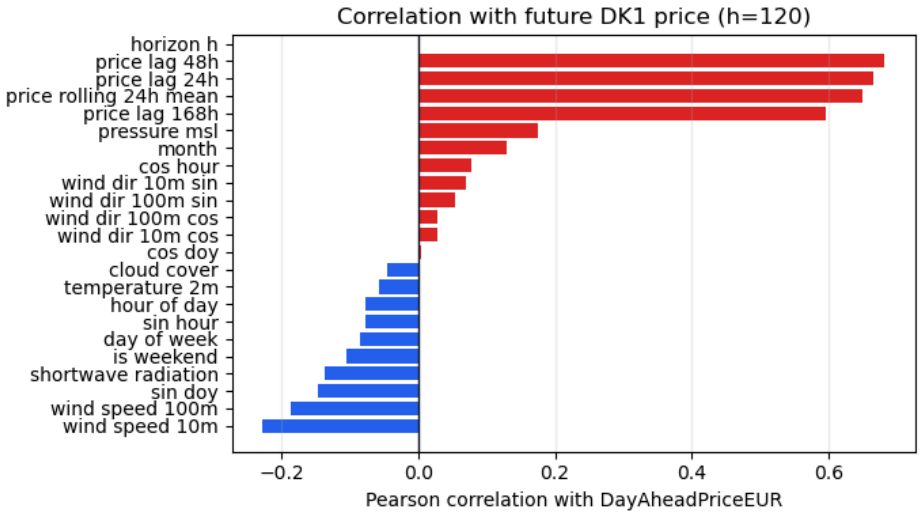
The weather plots show why the processing step is necessary. At a 24-hour horizon, the generated forecast-like weather variables generally follow the realised weather closely. At a 120-hour horizon, the deviations and uncertainty bands are wider. This is expected: longer weather forecasts are harder, and that uncertainty can propagate into electricity-price forecasts.

The implication for the recommender system is practical. A five-day forecast can still be valuable, but recommendations based on later horizons should be interpreted with more caution than recommendations based on the next few hours. This is one reason the forecast section evaluates both point accuracy and residual uncertainty by horizon.

Next examine which variables appear most informative for predicting future DK1 prices. For this purpose, we calculate the Pearson correlation between each feature and the target variable at the 120-hour horizon. This gives an initial indication of the strength and direction of the linear relationship between the input variables and the future price before fitting the forecasting models.

```
from src.analysis.forecast_analysis import plot_weather_price_correlation, plot_price_history_with_5day_forecast

fig = plot_weather_price_correlation(correlation_horizon=120)
```



The correlation plot is not a causal analysis but a sanity check â€” it confirms that the weather forecast variables and lagged price features carry statistical information about the future price target before any model is trained. Some relationships are likely nonlinear or context-dependent. Higher wind production, for example, tends to push prices down by increasing renewable supply, but the actual effect depends on demand, interconnector flows, and overall market conditions at the time. The variables shown here are exactly the features passed to the XGBoost model, and the full feature matrix is saved as`data/model_dataset.parquet`:

XGBoost Forecasting Model

XGBoost is a good choice for this project because electricity prices are tabular, nonlinear, and interaction-heavy. Gradient-boosted trees can capture threshold effects, such as very low wind or peak-hour demand, without requiring all interactions to be manually specified. The method also works well with mixed calendar, lag, and weather features, requires little feature scaling, and can later be inspected using feature importance and SHAP. Finally, the model is fast enough to retrain and evaluate with walk-forward validation, which is important for a time-series forecasting problem. A Temporal Fusion Transformer was also tested, but it performed poorly compared with XGBoost. This is likely because the available dataset is relatively limited and the forecasting problem is mainly based on structured tabular features.

The forecasting model is implemented in `src/models/XG_Boost_full_Res.py`. The script trains five separate XGBoost regression models, one for each forecast day:

- Day 1: horizons 1-24 hours ahead.
- Day 2: horizons 25-48 hours ahead.
- Day 3: horizons 49-72 hours ahead.
- Day 4: horizons 73-96 hours ahead.
- Day 5: horizons 97-120 hours ahead.

This five model design is useful because short-horizon and long-horizon forecasts have different error structures. The next few hours are usually more anchored by recent prices and near-term weather forecasts, while later horizons are more uncertain and depend more on broader calendar and weather-pattern information. Splitting the model by forecast day lets each XGBoost regressor specialize in one part of the 120-hour horizon.

After implementing the forecast model, validation is performed with an expanding walk-forward split. The first 12 months are used for training, the next 3 months are used for testing, and the training window then expands forward. This preserves the time order of the data and produces out-of-sample residuals that can be used both for accuracy metrics and uncertainty estimation.

The model performance is evaluated using MAE, since it measures the average forecast error in the same unit as the electricity price. It is easy to interpret and less affected by extreme price spikes than RMSE, making it suitable for assessing typical forecast accuracy.

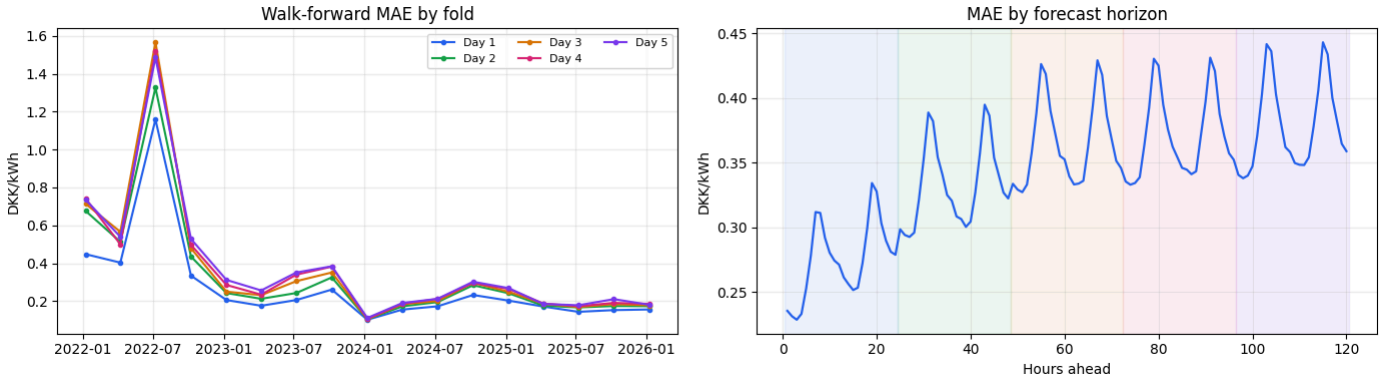
In the plot below, we can se how accurate the 5-day price forecasts are compared to the actual prices.

```
fig = plot_price_history_with_5day_forecast(
    issue_time="2026-04-23 00:00",
    ctx_days=7,
    unit="dkk_kwh",
    show_actuals=True,
    show_uncertainty=False,
)
```

The plot shows the historical DK1 price followed by the 5-day XGBoost forecast. The model captures the overall trends reasonably well, including the daily price pattern, but smooths some of the largest spikes and sudden drops. Since the model is evaluated using MAE, the focus is on typical forecast accuracy rather than heavily penalizing rare extreme price movements. A more general analysis of the model performance can be seen below:

```
from src.analysis.forecast_analysis import plot_forecast_mae_summary

fig = plot_forecast_mae_summary(unit="dkk_kwh")
```



The first plot shows how the different forecast horizons perform over time. It is clear that all models are affected by the energy crisis period, where the MAE increases sharply. This is expected because electricity prices became much harder to predict during this period due to high volatility and sudden price spikes. Overall, the shorter forecast horizons perform better, with Day 1 generally having the lowest error, while the longer horizons tend to have higher MAE. Second, the horizon plot shows how error develops across the 120-hour forecast window. This is directly relevant for recommendations: a predicted saving tomorrow is usually a stronger signal than a similarly sized predicted saving five days ahead if the five-day residual spread is much larger.

The forecast does not need to be perfect to be useful. For household flexibility, the key question is whether the model can rank future hours well enough to identify cheaper and more expensive windows. However, the size of the forecast error determines how forcefully the dashboard should communicate a recommendation. Small expected savings inside a wide error band should be framed cautiously, while large savings that exceed historical uncertainty can support a stronger recommendation.

Price Prediction Uncertainty: Residual-based forecast bands

The lecture material on prediction uncertainty emphasizes that a prediction should not be treated only as a single expected value. A future price is uncertain, so a decision-support system should also communicate the spread around the point forecast. In this project we estimate uncertainty from the walk-forward residuals, because those residuals measure how wrong the model was on data that was not used for fitting.

For each out-of-sample prediction we compute the signed residual

$$

e_h = \hat{y}_h - y_h,

$$

where $\hat{y}_h$ is the predicted price for horizon h and y_h is the realised price. We then group residuals by `horizon_h` and calculate a horizon-specific standard deviation, σ_h. The simple forecast band used here is

$$

\hat{y}_h \pm \sigma_h.

$$

The diagnostics below use residuals from 1 January 2023 onward. This keeps the uncertainty estimate focused on the post-energy-crisis market regime, rather than letting the extreme 2022 volatility dominate the current dashboard bands. The band should be interpreted as an empirical prediction-uncertainty measure, not as a guarantee that prices will fall inside the interval.

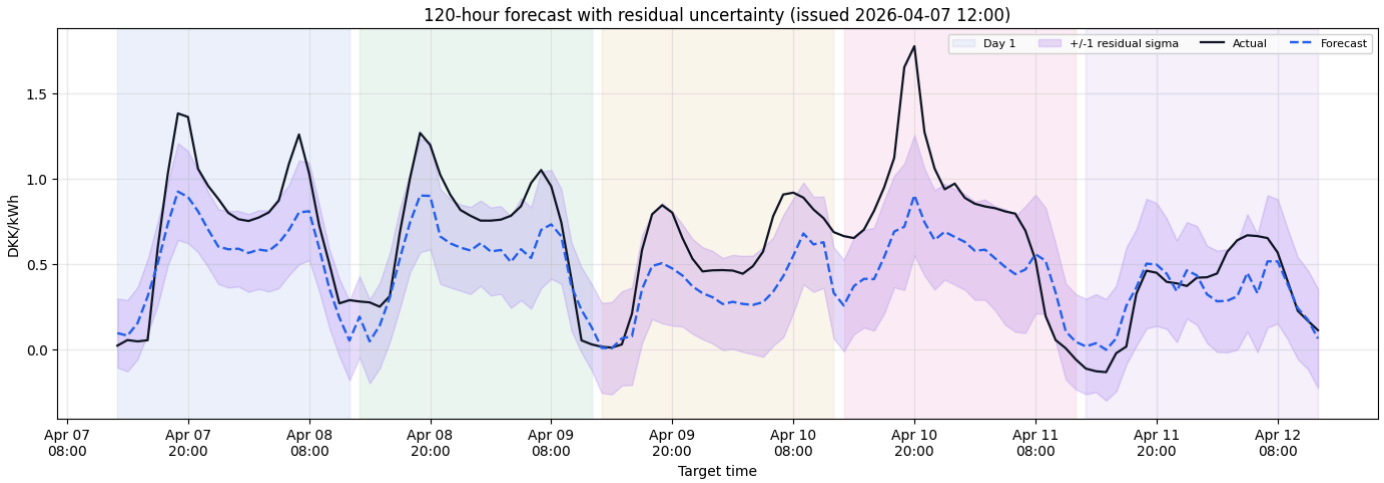
Below we can see how the user will see it in their dashboard:

```
from src.analysis.forecast_analysis import (
    plot_price_history_with_5day_forecast,
    plot_single_forecast_with_uncertainty,
)

issue_time = "2026-04-23 00:00"

fig_context = plot_price_history_with_5day_forecast(
    issue_time=issue_time,
    ctx_days=7,
    unit="dkk_kwh",
    show_actuals=True,
    show_uncertainty=True,
)

fig_uncertainty = plot_single_forecast_with_uncertainty(
    issue_time=issue_time,
    unit="dkk_kwh",
)
```



The uncertainty bands show how uncertain the predictions are. In this case, the bands are quite wide, which indicates that the model is still relatively uncertain about the exact future price level. This is partly a natural result of the data format and the limited information available to the model. In a final product, with more data, better feature engineering, and further model development, the predictions would likely become more certain.

The uncertainty is still useful because it makes the forecast easier to interpret. Instead of only showing a single predicted price, the dashboard also shows how much confidence the model has in that prediction. If the predicted cheapest period is only slightly cheaper than the current price and the uncertainty band is wide, the user should treat the recommendation with caution. If the predicted price drop is larger and the uncertainty band is narrower, the recommendation to delay flexible consumption becomes more reliable.

This also gives the consumer more control. By seeing the uncertainty directly in the dashboard, they can decide how much trust to place in the prediction. In this way, the uncertainty does not make the forecast less useful. It helps communicate when the modelâ€™s advice should be trusted strongly, and when it should be seen as more uncertain.

Explainable AI: SHAP and LIME for a decision-support forecast

The forecasting model produces a 120-hour price profile for DK1. The recommendation layer then translates that forecast profile into consumption advice: whether a household should consume energy now to avoid an expensive period or wait for a cheaper window. Explainable AI sits between these two steps. Its purpose is to analyse which input variables are driving the predicted prices and how. It is then fed into the LLM recommendation layer to make meaningfull and trustworthy reasoning.

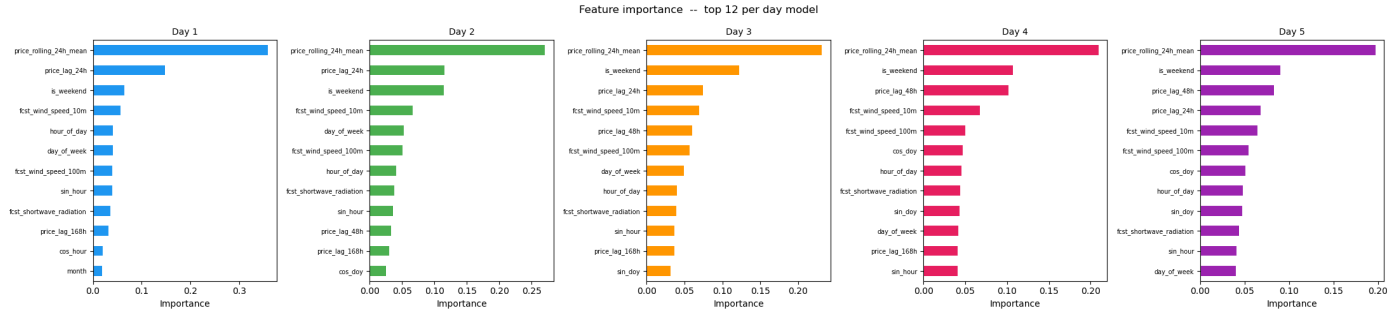
```
import joblib

from src.analysis.explainable_ai import (
    OUTPUT_DIR,
    make_dashboard_lime_summary,
    make_dashboard_shap_summary,
    plot_feature_importance,
    plot_lime_explanations,
    plot_shap_explanations,
)

final_models = joblib.load(OUTPUT_DIR / "final_day_models.joblib")
```

We start with XGBoost feature importance as a coarse global view. The model is split into five day-specific regressors, so the plot shows whether the short-horizon and long-horizon forecasts depend on similar drivers or whether the dominant predictors change across the 120-hour forecast window.

```
plot_feature_importance(final_models)
```



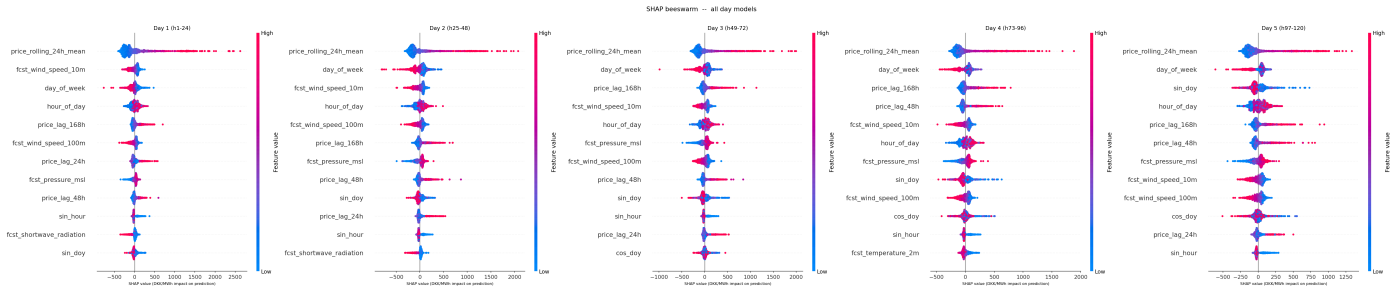
Across all five day-specific models, the rolling 24-hour mean price is the most important feature by a large margin. This means that the forecast is anchored heavily in the recent price level: if prices have been high during the last day, the model tends to expect elevated prices to continue, and if prices have recently been low, the forecast is pulled downward. This is a plausible result for electricity price forecasting. Day-ahead prices are not random from hour to hour; they often move in regimes where fuel prices, wind conditions, demand, imports/exports, and market expectations keep prices elevated or depressed for several consecutive hours or days. The model therefore appears to learn a meaningful persistence effect rather than relying only on weather forecasts.

Weather variables, especially fcst_wind_speed_10m and fcst_wind_speed_100m, appear among the top predictors across the forecast window, but they have lower importance than recent price features. This suggests that weather forecasts refine the price forecast around a baseline mainly set by recent market behaviour.

There is also a visible horizon effect. For Day 1, the model relies strongly on price_lag_24h, meaning yesterdayâ€™s mean price is highly informative for the next 24 hours. From Day 3 onward, price lags such as price_lag_24h, price_lag_48h, and price_lag_168h become more visible. This makes intuitive sense: as the forecast horizon expands, the exact price level from yesterday becomes less directly useful, while weekly seasonality and broader price regimes become more relevant. The increasing role of is_weekend, day_of_week, and time-cycle variables also supports this interpretation.

Feature importance is useful as an entry point, but it does not show whether a feature pushes the forecast up or down. It can also blur correlated drivers. We therefore use SHAP as the main global explanation method. SHAP decomposes each forecast into additive feature contributions, which is useful when the dashboard recommends a timing decision and we want to understand why a cheap or expensive window appears in the forecast.

```
plot_shap_explanations(width=1200)
```



The SHAP beeswarm plot gives a more detailed view of how the model uses each feature across many forecasts. Each point represents one forecasted hour, where the horizontal position shows whether the feature pushed the prediction up or down. The colour shows the feature value: red points are high feature values and blue points are low feature values.

The clearest pattern is again that price_rolling_24h_mean dominates the forecast across all five days. High recent average prices, shown by red points, are mostly placed on the positive side of the SHAP axis. This means that when the recent 24-hour price level is high, the model generally pushes the forecast upward. Low recent prices, shown by blue points, are mostly on the negative side, meaning that they pull the forecast downward. This is one of the most interpretable results in the Explainable AI section: the model has learned price persistence in a very direct way. The beeswarm plot also shows that this persistence effect remains important throughout the full 120-hour horizon, although the spread becomes smaller toward Day 5.

Several lagged price variables show similar, but weaker, effects. price_lag_168h becomes especially visible from Day 3 onward, indicating that the model uses prices from the same time one week earlier as a longer-horizon reference point. This supports the idea that the model combines short-term price persistence with weekly market patterns.

Wind variables also matter. For both fcst_wind_speed_10m and fcst_wind_speed_100m, lower wind values are often associated with positive SHAP values, meaning they push the predicted price upward. Higher wind values more often appear on the negative side, meaning they pull the predicted price downward. This fits the DK1 market logic: when wind conditions are weak, less wind generation is expected and price pressure can increase; when wind conditions are strong, higher renewable supply can help reduce prices.

Calendar variables such as day_of_week, hour_of_day, sin_doy, cos_doy, and sin_hour become more visible in the later horizons. These features capture recurring daily and weekly patterns rather than specific market shocks. Their importance supports the interpretation that, as the forecast moves further away from the present, the model relies more on regular time structure and less on very recent price observations.

For the project objective, households do not need to read feature importance plots or beeswarm SHAP plots before using the app. The SHAP values are used as input for the dashboard LLM recommendation layer. Thereby it allows it to analyse the relationship between the forecast predictions and the predictors because recommendation to e.g. wait should be traceable to sensible drivers such as renewable availability, recent price persistence, demand-sensitive weather, or time-of-day effects.

```
dashboard_shap_context = (
    make_dashboard_shap_summary(top_n=8)
    .loc[:, [
        "display_feature",
        "mean_abs_shap_dkk_kwh",
        "low_value_mean_shap_dkk_kwh",
        "high_value_mean_shap_dkk_kwh",
        "low_value_pressure",
        "high_value_pressure",
        "direction_summary",
    ]]
    .rename(columns={
        "display_feature": "feature",
        "mean_abs_shap_dkk_kwh": "global_strength_dkk_kwh",
        "low_value_mean_shap_dkk_kwh": "low_values_push_dkk_kwh",
        "high_value_mean_shap_dkk_kwh": "high_values_push_dkk_kwh",
    })
)

dashboard_shap_context.style.format(
    {
        "global_strength_dkk_kwh": "{:.4f}",
        "low_values_push_dkk_kwh": "{:+.4f}",
        "high_values_push_dkk_kwh": "{:+.4f}",
    }
)
```

	feature	global_strength_dkk_kwh	low_values_push_dkk_kwh	high_values_push_dkk_kwh	low_value_pressure	high_value_pressure	direction_summary
0	Recent 24h price average	0.2830	-0.2723	+0.6113	lowers predicted prices	raises predicted prices	high values push prices higher than low values
1	Forecast wind speed 10m	0.0749	+0.0910	-0.1292	raises predicted prices	lowers predicted prices	low values push prices higher than high values
2	Day of week	0.0726	+0.1022	-0.1079	raises predicted prices	lowers predicted prices	low values push prices higher than high values
3	Hour of day	0.0662	-0.0815	+0.0855	lowers predicted prices	raises predicted prices	high values push prices higher than low values
4	Price 168h ago	0.0597	-0.0497	+0.1398	lowers predicted prices	raises predicted prices	high values push prices higher than low values
5	Forecast wind speed 100m	0.0564	+0.0580	-0.1033	raises predicted prices	lowers predicted prices	low values push prices higher than high values
6	Price 24h ago	0.0520	+0.0319	+0.0489	raises predicted prices	raises predicted prices	high values push prices higher than low values
7	Price 48h ago	0.0438	-0.0366	+0.1074	lowers predicted prices	raises predicted prices	high values push prices higher than low values

The table above is the Day 1 SHAP export that is loaded by `dashboard.py` from `outputs/model/shap_day1_values.parquet` and `outputs/model/shap_day1_features.parquet`.

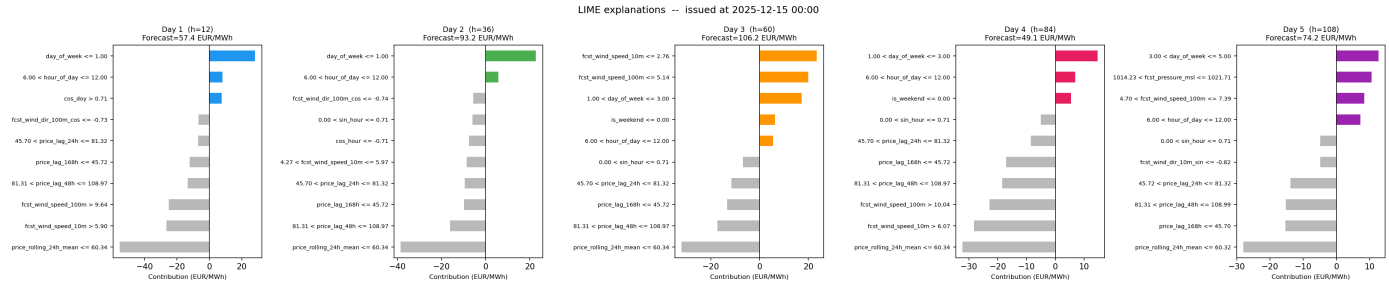
The LLM prompt receives a compressed text version of the same global model diagnostics.

The global model diagnostics compares the mean SHAP effect in the lower and upper feature-value quantiles. This preserves the beeswarm interpretation in a table-friendly form: for wind, low wind values tend to push prices upward while high wind values tend to pull prices downward.

In the LLM prompt, this becomes a small `MODEL DIAGNOSTICS` block. It is explicitly labelled as secondary evidence about what the XGBoost model has learned, not as causal truth. The LLM is instructed to prefer the actual forecast, intraday price pattern, and weather data whenever they conflict with the attribution summary. This turns SHAP into a communication layer: it helps the generated explanation stay connected to recognizable market drivers without making Explainable AI the only basis for the recommendation.

```
plot_lime_explanations(width=1200)
```

```
Loading model-ready dataset...
  463,784 rows | region=DK1_west
  After feature NaN drop: 463,784 rows
  Usable range: 2021-01-08 to 2026-04-24
  fig7 saved
```



LIME complements SHAP by focusing on one concrete forecast issue time. It fits a simple local surrogate around the selected forecast instance and shows which features pushed that specific prediction upward or downward. In other words, SHAP answers **"what generally drives this model?"**, while LIME helps answer **"why did the model make this recommendation now?"**

This local view is important for a consumer recommendation system because the same global model can produce different advice on different days. On one day, the recommended cheap window may be explained by strong wind and low recent prices. On another, a warning to consume earlier may be explained by lower renewable availability, higher lagged prices, and a peak-hour calendar effect.

The dashboard uses this idea in a deliberately narrow way. It does not send LIME explanations for all 120 forecast hours. Instead, it identifies the cheapest rolling 3-hour window and the most expensive rolling 3-hour window, then explains one representative hour inside each window. This keeps local Explainable AI focused on the actual decision points: when to shift flexible demand, and when to avoid consumption.

```
dashboard_lime_context = make_dashboard_lime_summary(final_models, top_n=4)
dashboard_lime_context[
    [
        "window_label",
        "window_start",
        "window_end",
        "window_avg_dkk_kwh",
        "representative_time",
        "horizon_h",
        "feature_rule",
        "direction",
        "contribution_dkk_kwh",
    ]
].style.format(
    {
        "window_start": lambda x: x.strftime("%a %d %b %H:%M"),
        "window_end": lambda x: x.strftime("%a %d %b %H:%M"),
        "window_avg_dkk_kwh": "{:.3f}",
        "representative_time": lambda x: x.strftime("%a %d %b %H:%M"),
        "contribution_dkk_kwh": "{:+.4f}",
    }
)
```

	window_label	window_start	window_end	window_avg_dkk_kwh	representative_time	horizon_h	feature_rule	direction	contribution_dkk_kwh
0	Cheapest selected window	Sat 11 Apr 13:00	Sat 11 Apr 16:00	0.017	Sat 11 Apr 14:00	98	60.91 < Recent 24h price average <= 87.96	lowers forecast	-0.2116
1	Cheapest selected window	Sat 11 Apr 13:00	Sat 11 Apr 16:00	0.017	Sat 11 Apr 14:00	98	Forecast wind speed 10m > 6.16	lowers forecast	-0.1649
2	Cheapest selected window	Sat 11 Apr 13:00	Sat 11 Apr 16:00	0.017	Sat 11 Apr 14:00	98	47.12 < Price 168h ago <= 81.60	lowers forecast	-0.1628
3	Cheapest selected window	Sat 11 Apr 13:00	Sat 11 Apr 16:00	0.017	Sat 11 Apr 14:00	98	Day-of-year cycle sine > 0.75	lowers forecast	-0.1243
4	Most expensive selected window	Tue 07 Apr 19:00	Tue 07 Apr 22:00	0.875	Tue 07 Apr 20:00	8	60.29 < Recent 24h price average <= 87.50	lowers forecast	-0.3211
5	Most expensive selected window	Tue 07 Apr 19:00	Tue 07 Apr 22:00	0.875	Tue 07 Apr 20:00	8	Forecast wind speed 10m <= 2.84	raises forecast	+0.1620
6	Most expensive selected window	Tue 07 Apr 19:00	Tue 07 Apr 22:00	0.875	Tue 07 Apr 20:00	8	Price 48h ago <= 45.72	lowers forecast	-0.1359
7	Most expensive selected window	Tue 07 Apr 19:00	Tue 07 Apr 22:00	0.875	Tue 07 Apr 20:00	8	Day of week <= 1.00	raises forecast	+0.1154

The table above mirrors the local LIME information passed to the dashboard LLM. Each row is one local contribution for a representative forecast hour inside either the selected cheap window or the selected expensive window. Positive contributions raise the local price forecast; negative contributions lower it. This is why LIME is useful for the dashboard's household guidance: it can explain why the current recommendation window looks cheap or risky without flooding the prompt with attribution values for every hour.

The dashboard computes this local attribution with `final_day_models.joblib` and matching rows from `data/model_dataset.parquet`. For historical issue dates from the walk-forward prediction file, the LIME values should therefore be read as a local diagnostic for the final model family, not as a perfect audit trail of the exact fold model that produced every stored prediction. This is another reason the LLM receives the diagnostics as secondary evidence rather than as the sole explanation.

This also matters for the final impact analysis. The shifting simulation estimates what happens when households react to forecast-based recommendations. Explainable AI does not prove that the behavioral impact will occur, but it makes the mechanism more credible: the simulated shifts are based on price signals that can be inspected and related back to recognizable market drivers. If the impact model shows large benefits while SHAP or LIME reveals that the forecast signal is driven by an implausible artifact, that would be a warning sign before interpreting the resilience contribution.

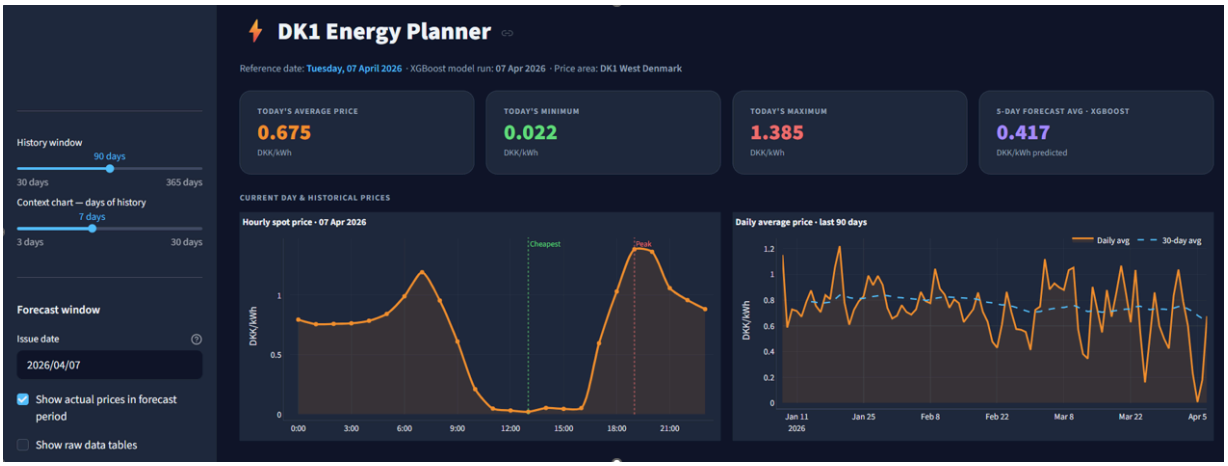
User Dashboard: Household Energy Recommendations

The dashboard brings together the full pipeline into a single interactive interface. It is built with Streamlit and is accessible at:

https://advanced-business-analytics-pwf6fcgrvyrfnhv9cq2pce.streamlit.app/

It combines the XGBoost price forecast, model explainability outputs, weather context, and an AI-generated reasoning summary. The goal is to translate model outputs into something a household can act on â€" showing when prices are expected to be high or low, why, and which hours are best for running flexible loads.

Dashboard overview



The user is met with a compact overview of the current and historical electricity price situation in DK1. At the top, key figures summarise todayâ€™s average, minimum, and maximum price, giving an immediate sense of the current price level. Below, the hourly spot price chart shows how prices change throughout the selected day, with the cheapest and most expensive hours highlighted.

Model insights: Forecast

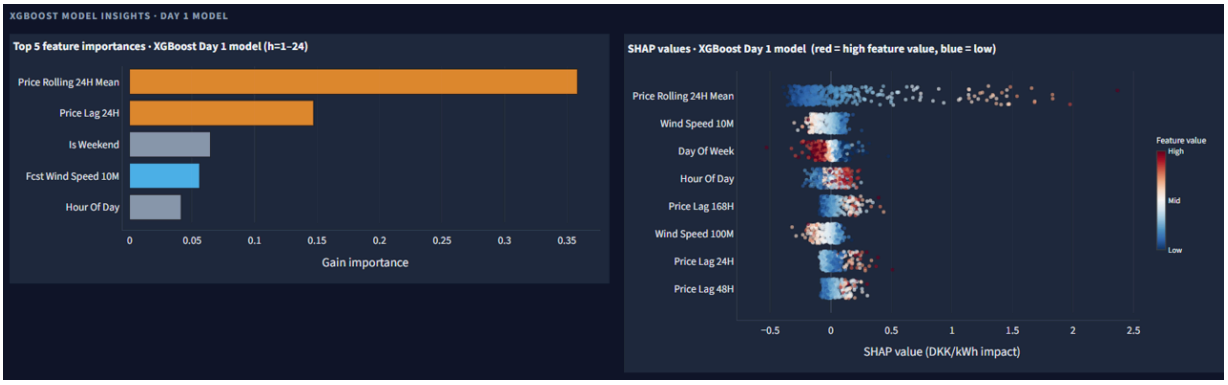
Following the introductory overview, the user is met with a 5-day spot price forecast for DK1. This section presents the expected daily average price for each of the coming days and compares the forecasted levels with todayâ€™s average price, making it easy to see whether prices are expected to rise or fall.

The hourly forecast chart above connects recent historical prices with the modelâ€™s 5-day forecast, creating a continuous view from observed price behaviour into the predicted period. In the sidebar, the user can adjust the history and context windows to choose which period should be shown around the forecast. The forecast can also be compared with actual prices when available, while the confidence band gives an indication of the uncertainty around the hourly predictions.

The actual prices can also be removed in the sidebar to illustrate what the dashboard would like when the actual price is not known.

Model explanations

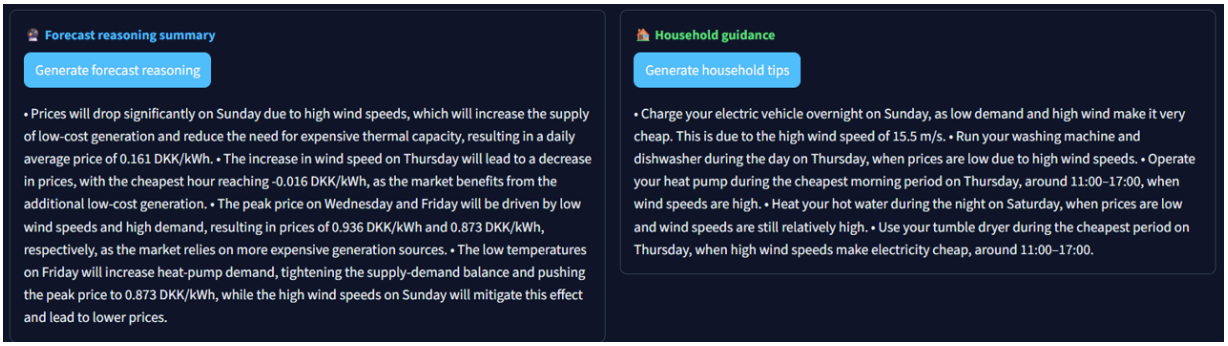
Explainable AI is used in the dashboard to keep the forecast and its interpretation connected. Instead of presenting the predicted prices as isolated outputs, the dashboard gives the user immediate access to the main factors behind the model behaviour at the point where the forecast is viewed.



This supports a more transparent workflow, where the user can move from observing the forecast to understanding which variables the model relied on most. In this way, the dashboard does not only communicate the expected price development, but also provides a built-in check of whether the model appears to base its predictions on meaningful and relevant patterns.

AI-generated reasoning

The dashboard includes two AI-generated summaries both triggered by a button.



The dashboard includes two Groq API calls that use LLM reasoning to translate the forecast into more understandable explanations for different users. These outputs are currently included as a proof of concept, showing how generative AI can be used to add interpretation and practical guidance on top of the numerical forecast.

Generate forecast reasoning is set up with a system prompt that tells Groq to write from the perspective of a Nordic electricity market analyst. The behaviour is guided through the system prompt and the information included in the API call. The dashboard sends the main forecast information, including predicted daily prices, the cheapest and most expensive hours, weather data, and model explanation outputs from SHAP and LIME. The LLM then reasons over these inputs to produce a short analytical explanation of the expected price development, rather than simply repeating the numbers shown in the dashboard. SHAP and LIME are used only as background information, so they help guide the reasoning.

Generate household tips is aimed at a Danish household. It uses the same price forecast together with the three cheapest time windows across the forecast period. The LLM reasons over the price pattern and identifies how flexible electricity use can be shifted toward cheaper periods. Groq then produces five practical tips for flexible loads such as EV charging, dishwashing, laundry, and heat pump operation. The tips group appliances around the cheapest periods and briefly explain why those time windows are favourable.

Both Groq API calls use Llama 3.3 70B with a temperature of 0.4 to keep the output consistent and focused. Each response is limited to a maximum of 650 tokens.

Modelling of impact: Rolling 120 hour energy shift heuristic

This section connects the forecasting and recommendation components directly to the resilience question. The objective is not only to assess how households potentially respond to price signals, but also to evaluate whether such responses meaningfully change the timing of demand in a way that reduces grid stress

To answer this, the project develops an impact-modelling framework that simulates how households may adjust their electricity consumption in response to forecast-guided recommendations. The simulated behavioural response is then used to adjust system level consumption loads. By comparing the original and adjusted load profiles, the framework assesses how the recommendation system could affect grid stress. These simulations therefore form the core basis for evaluating the potential resilience contribution of the project

Energy shift heuristic

As illustrated in the figure below, the impact is driven by many decentralised decisions made by households under uncertainty, constraints, and flexibility limits. Modelling such behaviour exactly would require detailed knowledge of user preferences, appliance availability, and real-time decision-making.

A heuristic approach is therefore appropriate, as it provides a **structured and transparent approximation of behaviour** under realistic constraints. Rather than assuming perfect optimisation, the heuristic captures key behavioural mechanisms:

- Households respond to **price signals**
- They are limited by **waiting times and flexibility constraints**
- They cannot shift all demand and must respect **capacity limits**

The heuristic thus represents a **bounded rational decision process**, where households shift consumption when it is beneficial, but only within realistic limits. This makes it well-suited for estimating system-level impact while maintaining interpretability and control over assumptions.

![Illustration of the rolling 120-hour energy shift heuristic](ABA_heuristic_illustration.png)

The figure outlines a **Step-by-step simulation of behaviour** on how the heuristic is applied across the simulation horizon to model household behaviour dynamically.

1. **Select issue times**

The simulation starts by selecting decision points where forecasts and recommendations are generated.

2. **Build a 120-hour window**

For each issue time, the model considers the next 120 hours of forecasted prices and actual system load. This defines the decision horizon available to the household.

3. **Create baseline and segments**

The household share is separated from the total system load and divided into behavioural segments: inflexible demand, wet loads, thermal demand, and EV charging.

Each segment is assigned a demand share, a maximum shiftable share, and a maximum waiting time.

4. **Evaluate candidate hours**

For each hour and each flexible segment, the model compares the current hour with future candidate hours inside the allowed waiting window.

Candidate hours with lower forecasted prices receive better scores, while longer waiting times are penalised.

5. **Shift energy subject to constraints**

If a future hour has a better score than the current hour, part of the flexible load is shifted to that hour.

The shift is only accepted if it respects the segment capacity, maximum waiting time, and maximum shiftable share.

6. **Repeat hour by hour**

The process is repeated sequentially across the 120-hour window. Over time, many small shifting decisions accumulate into a modified household and system load profile.

7. **Measure resilience impact**

The shifted load profile is compared with the baseline load profile. The resilience effect is measured by how much cumulative load above the historical p95 peak threshold is reduced.

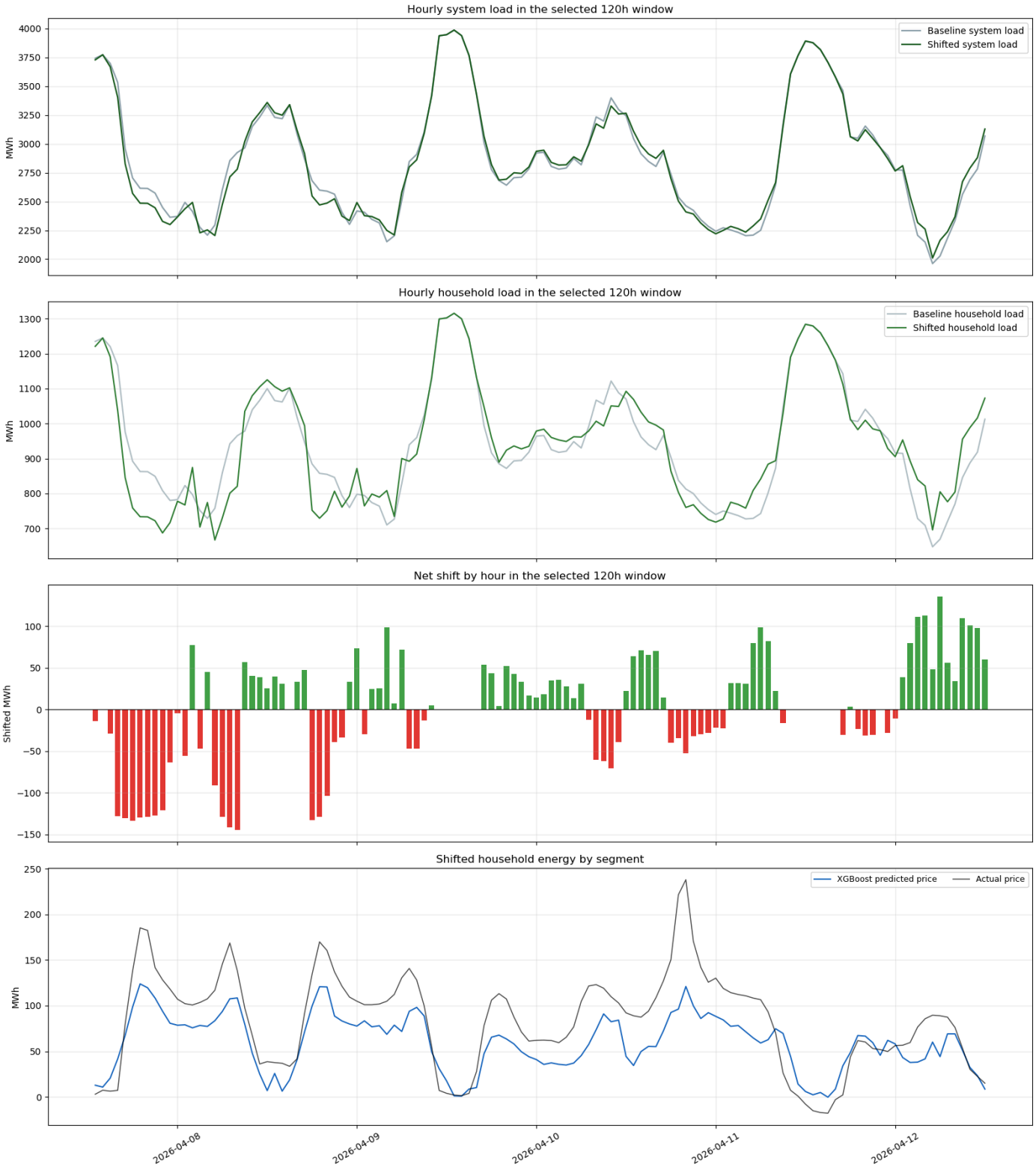
90-day simulation of shifted consumption

After the heuristic is defined, we simulate its effect hour for hour. For each selected issue time, the model decides what flexible energy to keep, what to move out of the current hour, and where to place it within the next 120 hours. The outputs are shifted household load, shifted system load, realized energy movement, and cost-based summary metrics.

To keep the impact section aligned with the forecasting evaluation, the simulation is run over the last 90 days. This is the same time span used as the final forecast test period. Because each forecast covers five days, we use non-overlapping 120-hour windows so that the 90-day impact estimate is not double-counting the same hours.

```
from src.analysis.impact_analysis import run_granular_shift_illustration

granular_result = run_granular_shift_illustration()
```



The four plots collectively illustrate how forecast-driven household behavior translates into measurable, but constrained, resilience improvements.

At the system and household load level (top two plots), the shifted profiles closely follow the baseline shape, confirming that total energy is preserved. The impact is therefore not a structural change in demand, but a temporal redistribution. The most visible effect occurs around peak periods, where the shifted load is slightly reduced and redistributed to adjacent hours. This demonstrates that the method successfully targets high-stress periods, but only partially flattens them.

The net shift plot (third panel) makes the behavioral mechanism explicit. Negative values (red) coincide with high-load periods, indicating that consumption is actively moved away from peaks, while positive values (green) appear in lower-load periods, where energy is reallocated. This reflects the intended optimization: households delay or advance flexible consumption within the allowed waiting window. The clustering of shifts suggests that behavior is not continuous, but occurs in discrete, opportunistic adjustments when favorable alternatives exist.

The price context (bottom plot) explains why these shifts occur. Periods with high actual and predicted prices align with negative shifts, confirming that households respond to price signals as intended. Conversely, periods with low or moderate prices show little or no shiftingâ€"even when system load is relatively high. This reveals a key behavioral constraint: shifting is driven by economic incentives, not directly by grid need.

The central insight on resilience impact:

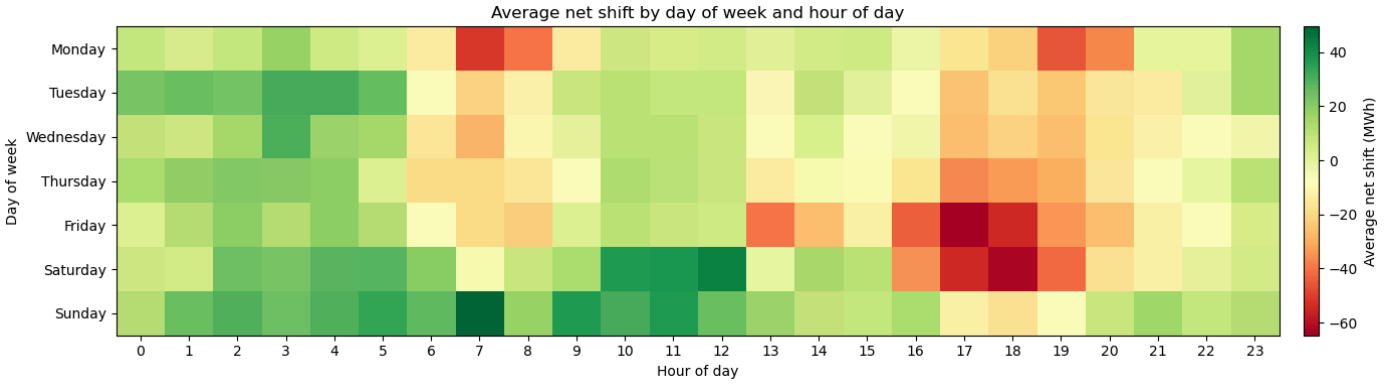
The method improves resilience by aligning flexible demand with price signals, which indirectly reduces pressure during some peak periods. However, the effect is selective rather than comprehensive. Peaks are reduced where price signals are strong, but persist where prices fail to reflect system stress. As a result, resilience gains are visible but moderate. The system benefits from smoother load profiles and reduced peak intensity, yet the remaining peaks highlight that price-based demand shifting alone cannot fully resolve grid pressure.

Behavior patterns across days and hours

While the previous figures illustrate how shifting reduces peak pressure within a specific time window, it is equally important to understand whether these effects follow systematic temporal patterns. The following heatmap aggregates the net shift across the full evaluation period by hour of day and day of week. This provides insight into when shifting occurs on average, and whether the heuristic consistently responds to recurring price signals rather than isolated events.

```
from src.analysis.impact_analysis import (
    plot_average_net_shift_heatmap,
    run_90_day_impact_simulation,
)

impact_90d = run_90_day_impact_simulation(simulation_days=90)
heatmap_df, _, _ = plot_average_net_shift_heatmap(impact_90d["profiles"])
```



The heatmap reveals a clear and structured pattern in shifting behavior, which supports both the validity of the heuristic and its contribution to resilience.

First, a consistent morning inflow of energy (green) is observed, particularly in the early hours (around 02:00â€"06:00) and again mid-morning. These periods typically correspond to lower electricity prices, indicating that the model systematically reallocates demand toward hours with favorable system conditions. In contrast, evening hours (approximately 17:00â€"20:00) show strong negative shifts (red), especially toward the end of the week. These hours coincide with well-known peak demand periods, where both consumption and prices tend to be high. The model therefore successfully identifies and avoids structurally stressed periods, rather than reacting randomly. This pattern is particularly pronounced on Fridays and Saturdays, where evening peaks are both larger and more consistently reduced. This suggests that the heuristic captures not only hourly dynamics but also weekly behavioral and price structures, which strengthens confidence in its general applicability.

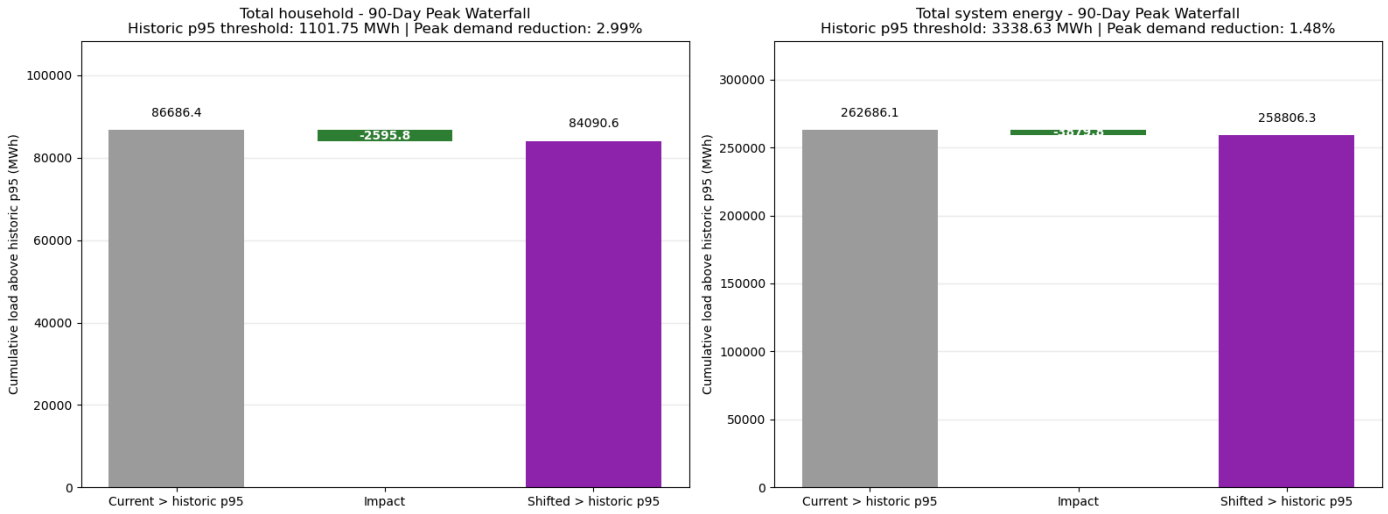
From a resilience perspective, this is important because it shows that the model contributes to predictable and repeatable load smoothing, rather than ad hoc adjustments. By consistently shifting demand away from recurring peak periods, the method helps reduce systemic stress in the hours where the grid is most vulnerable. At the same time, the heatmap also highlights limitations. Not all peak periods are fully mitigated, and some hours show only modest or no shifting. This indicates that the heuristic is constrained by the availability of economically attractive alternatives and the imposed waiting-time limits.

Overall simulated impact:

The heatmap demonstrated that the shifting heuristic produces consistent and structured behavioral patterns across hours and days, systematically moving demand away from recurring peak periods. However, while these temporal patterns validate how the model behaves, they do not directly quantify how much this behavior contributes to resilience. To assess the overall system-level impact, the following figure aggregates the effect across the full 90-day horizon, measuring the reduction in load above the historical p95 threshold.

```
from src.analysis.impact_analysis import plot_peak_excess_waterfalls

peak_summary_df, _, _ = plot_peak_excess_waterfalls(
    impact_90d["profiles"],
    impact_90d["system_load"],
)
```



At the household level, the model reduces cumulative load above the p95 threshold by 3.01%, corresponding to a reduction of approximately 2,614 MWh. This confirms that forecast-guided shifting successfully targets the most extreme consumption periods and redistributes a meaningful share of flexible demand.

At the system level, the corresponding reduction is 1.48%, reflecting that households represent only a fraction of total demand. While smaller in relative terms, this result is important: it demonstrates that **local behavioral adjustments can scale to system-level effects, even when applied to a limited share of consumption.**

Importantly, the waterfall structure highlights that the reduction is achieved without lowering total energy consumption. The impact arises purely from reducing the magnitude and duration of peak exceedances, rather than eliminating demand. This reinforces the interpretation of the method as a load-shaping mechanism rather than a demand-reduction strategy.

From a resilience perspective, this reduction implies that the system spends less time in its most stressed operating regime. Even moderate reductions in peak pressure can improve system stability, reduce the need for costly balancing actions, and enable better integration of renewable energy.

Assumptions behind the heuristic

The impact modelling relies on a set of explicit assumptions that define how much and how flexibly household demand can respond to price signals. These assumptions primarily act as modelling parameters, specifying the household share of system demand, the segmentation of consumption, the maximum shiftable shares, and the allowed waiting times.

The values used are informed by existing literature on household electricity consumption and demand-side flexibility, and are chosen to reflect a realistic and conservative representation of household behavior in energy systems. At the same time, they should not be interpreted as exact behavioral truths, but rather as scenario-based approximations that enable structured analysis.

Importantly, the assumptions extend beyond numerical parameters. They also include modelling choices about behavior, such as how households trade off price savings against waiting time, and how decisions are made within constrained flexibility windows. These choices define the decision logic of the heuristic and therefore directly shape the resulting system impact.

As such, the impact model should be understood as a proof-of-concept framework, where the purpose is not to perfectly predict real-world behavior, but to test whether forecast-guided demand shifting can plausibly contribute to system resilience under realistic constraints.

```
from src.analysis.impact_analysis import display_heuristic_assumption_tables

display_heuristic_assumption_tables()
```

```
<IPython.core.display.Markdown object>
```

```
<IPython.core.display.Markdown object>
```

The tables highlight that the model is intentionally conservative in its representation of flexibility. A substantial share of household demand is assumed to be inflexible, and even within flexible segments, only a limited portion can be shifted.

Flexibility is unevenly distributed across segments. EV charging exhibits the highest degree of flexibility, with both a large shiftable share and a long waiting window, while wet loads and thermal demand are more constrained. This reflects realistic differences in how long different energy services can be delayed without affecting user comfort.

The assumptions also embed a behavioral structure through the waiting-time penalties. These penalties reflect that households do not optimize purely for price, but balance cost savings against inconvenience. As a result, the model captures a form of bounded rationality, where shifting occurs only when it is sufficiently attractive relative to the required delay.

Together, these assumptions define the feasible action space of the heuristic. They determine when shifting is possible, how far it can occur in time, and how much energy can be moved. Changing these parameters would therefore directly alter the magnitude and timing of the simulated impact.

These assumptions play a central role in interpreting the results of the impact simulation. On one hand, the conservative setup strengthens the credibility of the findings: if measurable peak reduction is achieved under limited flexibility, it suggests that the mechanism is robust.

On the other hand, the assumptions also define the upper bound of the estimated impact. The model does not explore the full theoretical flexibility of the system, but rather a constrained and realistic subset. This means that the reported results likely represent a lower-bound estimate of what could be achieved under higher flexibility adoption or stronger behavioral incentives.

More broadly, the assumptions highlight that the resilience impact is not only a function of the forecasting model, but equally a function of how behavior is modelled. The simulation therefore reflects an interaction between price signals, behavioral response, and structural constraints, rather than a purely technical optimization outcome.

Discussion of resilience impact and limitation to the approach

This project shows how intelligent methods can support electricity-system resilience by translating price forecasts into actionable household recommendations. The value comes from the flexibility already present in household demand. Instead of relying only on physical grid expansion, which is costly and slow, the project explores how a lightweight digital solution can help move some consumption away from high-pressure periods and towards hours where electricity is expected to be cheaper and more abundant.

The simulation results indicate a measurable but moderate effect. Over the 90-day period, the model reduces household peak load by 3.01% and total system peak load by 1.48%. This does not mean that household shifting alone can solve grid congestion. Rather, it shows that forecasting and data driven recommendations can contribute to a broader resilience strategy. Small changes at household level can become meaningful when aggregated across many users, especially because the solution can be deployed relatively cheaply compared with major infrastructure upgrades.

Since direct grid-congestion data was not available, historical DK1 electricity consumption was used as a proxy for grid stress. Hours above the historical 95th percentile were interpreted as peak-pressure hours. This makes the analysis transparent and reproducible, but it is still an approximation. Actual grid stress depends on local network conditions, transformer capacity, voltage levels, and distribution-level bottlenecks. Therefore, the estimated impact should be understood as an indication of system-level resilience potential, not as a direct measure of physical congestion relief.

A key limitation is that household behavior is simulated rather than observed. The model assumes that some demand is flexible, that users respond to price-based recommendations, and that willingness to shift decreases as waiting time increases. It also relies on assumed household-load categories, such as wet loads, thermal load, EV charging, and inflexible demand, each with different shiftable shares and waiting limits. These assumptions make the impact mechanism testable and reproduceable, but they also determine the size of the estimated effect. The 3.01% household peak-load reduction should therefore not be interpreted as a final result, but as the result of a specific scenario. Real impact could be higher or lower depending on the complete set of assumptions. Furthermore, the strength of the impact of our solution could be heavily amplified if smart appliances, heat-pump and EV adoption increased. It would increase both the share of flexible demand and the general level of total system consumption which inflates the number of peak-hours. Future work should test sensitivity across low, medium, and high adoption scenarios.

The current shifting heuristic could be extended. For now, it moves consumption away from expensive or stressed hours into cheaper future hours. It does not support anticipatory behavior, where households could frontload because prices are expected to increase later. For example, an EV could be charged now if the next days are expected to be expensive or grid-stressed. Including both delaying and frontloading would make the system more realistic and potentially strengthen the impact.

The XGBoost is suitable for nonlinear tabular forecasting and supports explainability, but the model is limited by the available predictors. Electricity prices are affected by factors beyond historical prices and weather, including renewable generation forecasts, demand forecasts, fuel prices, interconnector flows, outages, and broader market conditions. Missing variables may reduce forecast accuracy, especially during unusual market situations. Since the impact simulation depends on forecasted prices, forecast errors can lead to weaker or even misplaced recommendations. A deployable version should therefore use richer real-time inputs and update continuously.

Despite these limitations, the project has clear practical value. It shows that a relatively simple digital layer can help consumers interact more intelligently with the electricity system. The system translates complex price and forecast signals into understandable recommendations, without requiring households to understand wholesale electricity markets or grid conditions. Compared with physical grid reinforcement, such a solution could be faster and cheaper to deploy, while still contributing to peak reduction and better use of periods with high supply and low demand. Its role is not to replace grid expansion, but to complement it by activating demand-side flexibility.

Conclusion

This project investigates whether intelligent methods can strengthen energy grid resilience in DK1 by guiding households to shift flexible consumption, thereby reducing grid stress in peak-hours.â€‹

The project shows that price forecasts can be translated into household recommendations that encourage flexible electricity use. The results show that forecast-guided household flexibility can create a measurable but moderate resilience effect. In simulation, household peak load was reduced by 3.01% and total system peak load by 1.48%, a meaningful but moderate impact that points to a realistic, data-driven pathway for supporting grid resilience. While household shifting alone is unlikely to resolve congestion, the results suggest it can serve as a valuable layer within a broader resilience strategy.

The forecasting results show that historical electricity prices and weather indicators provide a useful basis for predicting future price patterns. However, the modelâ€™s accuracy is still limited by the available predictors. A more reliable and deployable system would require richer inputs, such as renewable generation forecasts, demand forecasts, fuel prices, interconnector capacity, and broader market conditions.â€‹

The LLM layer, combined with explainable AI, helps make the system more understandable and user-friendly. Our dashboard

demonstrates how an LLM can be used together with Explainable AI to make the forecast results easier to understand and

more actionable for the user. â€‹

â€‹

A real-world application would need further validation. In this project, household behavior is simulated, and DK1 system consumption is only a proxy for local grid stress. Future work should therefore test whether real households follow the recommendations, use local grid data, and include more direct information about renewable production and expected electricity demand.â€‹